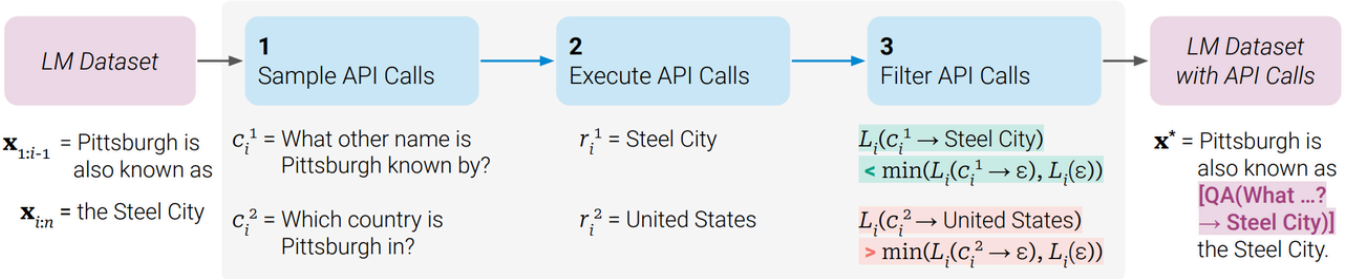
# Toolformer: Language Models Can Teach Themselves to Use Tools

> #Tools #Finetuning




- What problems does this framework solve?
  - Though LLMs perform well on 0-shots and few-shots tasks such as common-sense QA, on some basic tasks, such as calculation, fact findings, and time sensation, their performance is not so desirable.
  - One way to improve their performance is guiding them to use tools. However, traditionally, this will either be expensive (human labeling) or be poor in generalization.
- What does this framework do?
  - Let `<API>`, `</API>`, and `->` be predefined special tokens. We use:

    `e(c, r) = <API> ac(ic) → r </API>`

    to represent an API call with return values, and

    `e(c) = <API> ac(ic) </API>`

    to represent an API call without return values.
  - Researchers will write short descriptions for each API. Given a normal training dataset, insert the API call at the place where the model has a probability to generate `<API>` greater than a threshold when it is browsing the text of the dataset. Sample $m$ trajectories.
  - Finish the trajectories by running the selected APIs.
  - Filtering: Calculate the CE loss of the following token generation
    - $L^+$: The following token generation's CE loss **with API and results** inserted
    - $L^-$ (baseline): The **minimum** value of: 1. No API calls 2. With API calls but without answers.
    - *Why Minimum? If the API helps, the CE loss will be lower. We want to find an $L^+$ that really helps the model, i.e., much better than the baseline. To make the comparison rigorous and strict, we select the best form the better's.*
    - Keep the $L^+$ such that $L_i^- - L_i^+ \geq \tau_f$.
  - Use the selected $L^+$'s to form a finetuning set. Finetune the model.
  - Inference: During inference, when the model generates `->` token, the generation will be paused and the API will be called. Then, the API results will be added to the context. Generation continues.
- Experiment Results

    - Significant improvement in zero-shot performance, outperforming models of higher capacity: Toolformer significantly outperforms GPT-J (without tools) and even OPT (66B)—which is several times larger—across multiple downstream tasks (particularly the Math LAMA and QA benchmarks), and its performance on several tasks rivals that of the massive GPT-3 (175B).
    - No compromise on core language capabilities: Experiments confirm that this fine-tuning method, which involves inserting API calls, does not impair the model's inherent language modeling capabilities; the model's perplexity on benchmarks such as WikiText remains unchanged.
    - Scaling Laws: Experiments reveal that a model's ability to effectively utilize tools does not emerge immediately but only begins to manifest when the number of parameters reaches approximately 775M. Smaller models, even when provided with tools, fail to learn how to use them effectively.
- Work Cited
    - Paper
    - Chat History